

Trabalho A3 — Estudo de Caso Aplicado
Estruturas de Dados e Análise de Algoritmos

**Spotify: Estruturas de Dados e Algoritmos em
Sistemas de Recomendação Musical**

Gustavo Nunes Cunha; 123113556
Leonardo Oliveira Moraes; 1242020286
Luis Filipe da Silva Periard; 1232021702
Lucas Gabriel Lopes de Souza; 12317548
Felipe dos Santos Marques; 123115320
Matheus Tetsuo Paiva Okano; 1221121320

Alexandre “Montanha” Oliveira

15/06/2026

1. Introdução

O crescimento da quantidade de conteúdo digital tornou a personalização uma necessidade para plataformas modernas. Serviços de streaming, redes sociais e e-commerce precisam ajudar usuários a encontrar conteúdos relevantes em meio a milhões de opções disponíveis.

Nesse contexto, o Spotify se destaca como uma plataforma de streaming de áudio que utiliza sistemas de recomendação para sugerir músicas, artistas, álbuns, podcasts e playlists personalizadas. Fundado em 2006, na Suécia, o Spotify está entre os principais serviços de música e podcasts do mundo, oferecendo acesso a um grande catálogo de conteúdos digitais.

Grande parte da experiência do usuário no Spotify depende de recomendações personalizadas, como playlists automáticas, rádios personalizadas e sugestões na tela inicial. Assim, a plataforma não atua apenas como um catálogo de músicas, mas como um sistema que organiza, filtra e recomenda conteúdos conforme o perfil e o momento de cada usuário.

O principal desafio computacional da plataforma é recomendar o conteúdo certo para o usuário certo, no momento certo. Esse problema é complexo porque envolve milhões de usuários, milhões de músicas e podcasts, grande volume de interações diárias, preferências diferentes, mudanças no gosto musical e necessidade de resposta rápida no aplicativo.

Uma comparação direta entre todos os usuários e todas as músicas seria inviável em larga escala. Considerando U como o número de usuários e M como o número de músicas, uma abordagem ingênua teria custo próximo de $O(U \times M)$, exigindo processamento excessivo e tempo de resposta inadequado.

Por isso, o Spotify precisa utilizar algoritmos e estruturas de dados capazes de reduzir o espaço de busca e encontrar boas recomendações com eficiência. A qualidade dessas recomendações impacta a satisfação dos usuários, a permanência na plataforma, a descoberta de novos artistas e a fidelização. Dessa forma, o problema é técnico e estratégico, tornando o caso relevante para a disciplina por envolver busca, otimização, escalabilidade, processamento de dados e tomada de decisão algorítmica.

2. Raio-X Técnico do Caso

2.1 Visão Geral Técnica

O sistema de recomendação do Spotify pode ser entendido como um fluxo de processamento de dados. A plataforma coleta interações dos usuários, processa essas informações, identifica padrões de comportamento e utiliza algoritmos para sugerir conteúdos com maior chance de interesse.

De forma simplificada, o fluxo ocorre assim:

usuário interage com o aplicativo → dados são coletados → candidatos são selecionados → recomendações são ranqueadas → conteúdo é exibido ao usuário.

Como o Spotify lida com milhões de usuários, músicas, podcasts e interações diárias, o sistema não pode analisar todo o catálogo do zero a cada acesso. Por isso, precisa reduzir o espaço de busca e entregar respostas rápidas.

2.2 Arquitetura e Stack Tecnológica

O Spotify utiliza uma arquitetura distribuída, composta por diferentes serviços que trabalham de forma integrada. Essa estrutura permite separar tarefas como coleta de dados, processamento, recomendação e entrega de conteúdo, melhorando a escalabilidade.

Entre os principais componentes de sistemas desse tipo estão:

aplicações web e mobile;

APIs de comunicação;

armazenamento distribuído;

filas de processamento;

cache;

modelos de aprendizado de máquina.

2.3 Problema Computacional Envolvido

O principal problema computacional do Spotify é selecionar, dentro de um catálogo muito grande, quais conteúdos são mais relevantes para cada usuário.

Para isso, o sistema considera informações como histórico de reprodução, músicas curtidas, skips, playlists criadas, comportamento de usuários semelhantes e características das músicas.

A dificuldade está no tamanho do espaço de busca. Pela notação Big O, uma comparação direta entre todos os usuários e todas as músicas teria complexidade aproximada de $O(U \times M)$, em que U representa usuários e M representa músicas. Conforme esses valores crescem, o número de comparações aumenta rapidamente, tornando a abordagem inviável.

Por isso, o sistema precisa selecionar candidatos mais prováveis e aplicar ranking sobre um conjunto menor de opções.

2.4 Escalabilidade e Complexidade

A escalabilidade é um dos principais desafios do caso. Conforme a plataforma cresce, também aumentam usuários, músicas, playlists, podcasts e interações. Mesmo assim, o sistema precisa continuar respondendo rapidamente.

Para tornar o problema viável, os sistemas de recomendação utilizam estratégias como seleção inicial de candidatos, vetores, filtragem colaborativa, filtragem baseada em conteúdo, cache, processamento distribuído e ranking.

Essas estratégias evitam comparações desnecessárias e mostram uma ideia importante da disciplina: em sistemas reais, uma solução suficientemente boa e rápida costuma ser mais viável do que uma solução teoricamente perfeita, mas lenta demais.

2.5 Classificação Conceitual do Problema

O caso do Spotify pode ser classificado como um problema de recomendação, busca, otimização e escalabilidade.

É um problema de recomendação porque busca sugerir conteúdos compatíveis com o perfil do usuário. É um problema de busca porque precisa encontrar conteúdos relevantes em um catálogo amplo. É um problema de otimização porque tenta maximizar relevância e engajamento com baixo tempo de resposta. Também é um problema de escalabilidade, pois precisa continuar eficiente com o crescimento dos dados.

Neste trabalho, o problema não será tratado como NP-Completo, pois o objetivo não é demonstrar uma solução ótima formal, mas analisar uma solução prática de recomendação em larga escala.

2.6 Estratégia Utilizada pelo Spotify

O Spotify combina diferentes estratégias para gerar recomendações personalizadas.

A filtragem colaborativa recomenda conteúdos com base em usuários de comportamento semelhante. A filtragem baseada em conteúdo considera características das músicas, como gênero, ritmo, energia, artista e estilo.

Além disso, a busca aproximada e o ranking ajudam a reduzir o espaço de busca e ordenar os candidatos mais relevantes. O cache evita recálculos desnecessários, enquanto o processamento distribuído permite lidar com grandes volumes de dados.

Dessa forma, a solução mostra que sistemas de recomendação em larga escala dependem da combinação entre algoritmos, estruturas de dados e decisões arquiteturais.

3. Relação com Estruturas de Dados

Para manter o trabalho objetivo, foram selecionadas três estruturas de dados principais que ajudam a explicar o funcionamento de sistemas de recomendação.

3.1 Matriz Usuário-Item

A matriz usuário-item representa a relação entre usuários e músicas. Cada linha pode representar um usuário, cada coluna uma música, e cada célula indica algum tipo de interação, como reprodução, curtida, skip ou adição a uma playlist.

Essa estrutura é importante para a filtragem colaborativa, pois permite identificar usuários com comportamentos parecidos.

Por exemplo, se dois usuários escutam muitas músicas em comum, o sistema pode recomendar para um usuário músicas que o outro já ouviu e gostou.

A limitação dessa estrutura é que ela tende a ser muito grande e esparsa, pois cada usuário consome apenas uma pequena parte do catálogo total.

3.2 Vetores e Embeddings

Vetores, ou embeddings, são usados para representar usuários e músicas em formato numérico. A ideia é transformar características e comportamentos em valores que possam ser comparados matematicamente.

Com essa representação, músicas parecidas ficam próximas em um espaço vetorial. Usuários com preferências semelhantes também podem ter vetores próximos.

Isso ajuda o sistema a encontrar recomendações sem precisar comparar diretamente todos os itens do catálogo. Em vez disso, ele procura músicas próximas ao perfil do usuário.

Essa estratégia reduz o custo computacional e melhora a eficiência da busca.

3.3 Cache e Filas

O cache é utilizado para armazenar temporariamente dados ou recomendações já calculadas. Isso reduz o tempo de resposta, pois o sistema não precisa recalculá-los a cada acesso.

As filas são utilizadas para processar eventos de forma assíncrona. Por exemplo, quando um usuário escuta uma música, pula uma faixa ou curte uma playlist, esse evento pode ser enviado para uma fila e processado posteriormente.

Essas estruturas ajudam a manter o desempenho da plataforma, evitando sobrecarga no sistema principal.

4. Algoritmos e Estratégias Utilizadas

4.1 Filtragem Colaborativa

A filtragem colaborativa recomenda conteúdos com base no comportamento de usuários semelhantes.

Se dois usuários possuem gostos parecidos, músicas apreciadas por um deles podem ser recomendadas ao outro. Essa estratégia é útil porque aproveita padrões coletivos de consumo.

A principal vantagem é descobrir recomendações a partir da experiência de outros usuários. A limitação é que ela depende de histórico de interação. Usuários novos ou músicas novas possuem poucos dados, o que dificulta a recomendação.

4.2 Filtragem Baseada em Conteúdo

A filtragem baseada em conteúdo utiliza características das próprias músicas para gerar recomendações.

Essas características podem incluir:

- * gênero;
- * ritmo;
- * energia;
- * artista;
- * estilo;
- * tonalidade;
- * características acústicas.

Essa abordagem permite recomendar músicas semelhantes às que o usuário já ouviu. Ela é útil principalmente quando há pouco histórico coletivo sobre determinada música.

A limitação é que o sistema pode acabar recomendando conteúdos muito parecidos, reduzindo a diversidade da experiência.

4.3 Busca Aproximada e Ranking

Depois de identificar possíveis candidatos, o sistema precisa ordenar as recomendações por relevância. Essa etapa é chamada de ranking.

Antes do ranking, porém, é necessário reduzir o espaço de busca. Em vez de analisar todas as músicas disponíveis, o sistema seleciona um conjunto menor de candidatos com maior chance de interesse.

A busca aproximada ajuda nesse processo. Ela não garante encontrar a melhor recomendação possível em termos matemáticos, mas encontra boas recomendações com menor custo computacional.

Essa troca entre precisão e desempenho é comum em sistemas reais de larga escala.

5. Proposta Alternativa do Grupo

5.1 Visão Geral da Proposta

Antes da proposta alternativa, é importante observar que o Spotify já utiliza diferentes estratégias de recomendação, considerando histórico musical, músicas curtidas ou puladas, playlists criadas, artistas seguidos, comportamento de usuários semelhantes, características das músicas e contexto recente de uso.

Com esses dados, a plataforma combina filtragem colaborativa, filtragem baseada em conteúdo, aprendizado de máquina, busca aproximada, ranking e cache para gerar recomendações personalizadas sem comparar todo o catálogo a cada solicitação.

Como proposta complementar, o grupo sugere uma camada contextual de recomendação, inspirada em sinais de comportamento usados em redes sociais, como o Instagram. A proposta não substituiria o sistema atual, mas adicionaria uma etapa para tornar as recomendações mais sensíveis ao momento do usuário.

A ideia central é combinar o histórico musical com sinais recentes de comportamento, sempre com autorização do usuário. Assim, o sistema poderia adaptar melhor as recomendações a contextos como estudo, atividade física, relaxamento, produtividade ou lazer.

5.2 Funcionamento da Proposta

A proposta funcionaria em quatro etapas principais.

Primeiro, ocorreria a coleta autorizada de sinais. O usuário poderia permitir o uso de informações comportamentais internas ou externas, como interesses recentes, categorias de conteúdo consumidas e padrões de interação. Esses dados seriam usados de forma agregada, sem necessidade de identificar exatamente cada conteúdo visualizado.

Depois, o sistema faria a classificação do contexto. A partir dos sinais recentes, o usuário poderia ser associado temporariamente a categorias como estudo, atividade física, relaxamento, produtividade, lazer ou viagem.

Em seguida, ocorreria a combinação com o histórico musical. O contexto identificado seria cruzado com o histórico de reprodução, músicas curtidas, artistas favoritos e preferências já conhecidas do usuário.

Por fim, seria feita a geração da recomendação. O sistema selecionaria candidatos compatíveis com o perfil e o contexto identificado, aplicando ranking para definir quais conteúdos seriam exibidos primeiro. Por exemplo, um usuário que costuma ouvir rock e está em contexto de atividade física poderia receber playlists de rock com maior energia.

5.3 Relação com Estruturas de Dados e Algoritmos

A proposta depende diretamente da forma como os dados são armazenados, processados e comparados.

Elemento	Aplicação na proposta
Matriz usuário-item	Armazena interações musicais, como reproduções, curtidas e skips
Vetores/embeddings	Representam usuários, músicas e contextos numericamente
Tabelas hash	Permitem acessar rapidamente perfis e categorias de contexto
Filas	Organizam eventos recentes para processamento assíncrono
Cache	Reaproveita recomendações ou contextos já calculados
Ranking	Ordena os candidatos mais relevantes

A matriz usuário-item representa o histórico musical, enquanto os vetores ou embeddings permitem comparar usuários, músicas e contextos por similaridade. As tabelas hash ajudam no acesso rápido aos dados do perfil, as filas processam eventos sem sobrecarregar o sistema principal e o cache reduz recálculos desnecessários.

5.4 Análise de Complexidade da Proposta

A proposta busca reduzir o custo computacional da recomendação. Em uma abordagem ingênua, comparar todos os usuários com todas as músicas teria custo próximo de $O(U \times M)$, em que U representa usuários e M representa músicas.

Com a camada contextual, o sistema reduziria o espaço de busca antes do ranking. Em vez de avaliar todo o catálogo, ele selecionaria apenas um subconjunto de músicas compatíveis com o perfil e o contexto do usuário.

De forma simplificada, o fluxo seria:

catálogo completo → filtro por contexto → candidatos relevantes → ranking final

Assim, o ranking passa a ocorrer sobre um conjunto menor de candidatos, representado por k , e não sobre todo o catálogo. Isso mostra a aplicação prática da análise de algoritmos: encontrar uma boa recomendação em tempo viável, sem avaliar todas as possibilidades.

5.5 Vantagens Computacionais da Proposta

A principal vantagem da proposta é reduzir o espaço de busca usando uma etapa contextual. Ao identificar o momento do usuário antes da recomendação, o sistema pode priorizar músicas e playlists mais compatíveis com aquele contexto.

Além disso, cache e filas ajudam no desempenho. O cache evita recálculos, enquanto as filas permitem processar eventos recentes de forma assíncrona, mantendo o sistema mais eficiente.

A proposta também equilibra histórico e comportamento recente. O histórico musical representa preferências consolidadas, enquanto o contexto recente ajuda a adaptar as recomendações ao momento atual do usuário.

6. Análise Crítica e Conclusão

A análise do caso Spotify mostra que sistemas de recomendação em larga escala precisam equilibrar precisão, desempenho, escalabilidade, custo computacional e experiência do usuário. Embora modelos mais complexos possam gerar recomendações mais precisas, a plataforma precisa entregar respostas rápidas e viáveis para milhões de usuários.

Do ponto de vista da disciplina, o principal aprendizado está na diferença entre uma solução teoricamente completa e uma solução prática. Comparar todos os usuários com todas as músicas poderia gerar recomendações mais detalhadas, mas teria custo computacional inviável. Por isso, sistemas reais utilizam estratégias como redução do espaço de busca, vetores, filtragem colaborativa, filtragem baseada em conteúdo, cache, filas e ranqueamento.

A proposta alternativa do grupo, baseada em uma camada contextual, busca complementar esse processo ao considerar sinais recentes de comportamento do usuário. Essa abordagem poderia tornar as recomendações mais sensíveis ao momento, equilibrando preferências históricas com contextos atuais, como estudo, atividade física, relaxamento e lazer.

Mesmo assim, a proposta exige cuidado. O uso de sinais comportamentais deve respeitar a autorização do usuário e a proteção dos dados. Além disso, nem todo comportamento recente representa uma preferência real, o que pode gerar recomendações pouco precisas se o sistema der peso excessivo ao contexto momentâneo.

Conclui-se que o Spotify representa um caso relevante de aplicação prática de Estruturas de Dados e Análise de Algoritmos. O estudo demonstra que conceitos como notação Big O, busca, otimização, cache, filas, vetores e ranqueamento são fundamentais para resolver problemas reais em plataformas digitais de grande escala.

Referências

SPOTIFY ENGINEERING. Big Data Processing at Spotify: The Road to Scio. Spotify Engineering Blog, 2017.

SPOTIFY RESEARCH. Contextual and Sequential User Embeddings for Music Recommendation. Spotify Research, 2021.

SPOTIFY FOR DEVELOPERS. Spotify Web API Documentation. Spotify Developer Platform.

SPOTIFY ENGINEERING. Why We Use Separate Tech Stacks for Personalization and Experimentation. Spotify Engineering Blog, 2026.

GOOGLE CLOUD. Spotify customer story: Open source data productivity from Spotify. Google Cloud Customers.

SÁNCHEZ-MORENO, Diego; ZHENG, Yong; MORENO-GARCÍA, María N. Time-Aware Music Recommender Systems: Modeling the Evolution of Implicit User Preferences and User Listening Habits in a Collaborative Filtering Approach. 2020.

RICCI, Francesco; ROKACH, Lior; SHAPIRA, Bracha. Recommender Systems Handbook. Springer.

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford. Algoritmos: Teoria e Prática. MIT Press / GEN LTC.

BRASIL. Lei nº 13.709/2018 — Lei Geral de Proteção de Dados Pessoais.